**Contribution report for project Deep Vision**

# Comparison of CNN vs. Vision Transformer for Image Classification with Transfer Learning from ImageNet for Small Dataset

**Muhammad Aaliyain, m.aaliyan@oth-aw.de**[0] **and Farrukh Rasool, f.rasool@oth-aw.de**[1]

[1]*OTH Amberg-Weiden, Study programme "Deep Vision"*

Report written on June 24, 2026

**Abstract**

**The project** presents a comparison between a Convolutional Neural Network (ResNet-50) and a Vision Transformer (ViT-B16) for image classification using transfer learning from ImageNet. The main goal is to understand how these two architectures perform on the image classification on small dataset using tensor-flow. Several datasets form official tfds were evaluated before selecting a 10 class subset of Food-101, as other options like Oxford Flowers 102, CIFAR-100, PlantVillage, and Oxford-IIIT Pets were found unsuitable for different reasons.
**This report** covers the complete experimental pipelines including dataset selection, preprocessing, model design, a full ResNet-50 study across six classification head variants, fine-tuning and a complete ViT-B16 comparison across five head configurations. ResNet-50 achieved a best test accuracy of 93% while ViT-B16 reached 95% on the same 10 class Food-101 test set, showing that Vision Transformers generalise better than CNNs under transfer learning on small datasets.

**Keywords:** *Convolutional Neural Network, Vision Transformer, TensorFlow*

## 1. Introduction

This report includes the comparison for two of architectures Convolutional Neural Networks (CNNs) [5] and Vision Transformers (ViTs)[6]. CNNs process images by detecting local features such as edges and textures through convolutional filters, while ViTs divide the image into patches and use self-attention to capture global relationships across the whole image.

The transfer learning technique is being used for both of the architectures by using weights pretrained on a large dataset like ImageNet and adapting them to a new task with small dataset to investigate how these models compete on a small dataset.

For this comparison, 10 classes from Food-101 was used as the target dataset, providing 7,500 training images. ResNet-50 was selected as the CNN architecture and ViT-B16 as the transformer. Multiple classification head variants were trained on the frozen ResNet-50 backbone to study the effect of different regularisation techniques before fine-tuning. The same pipeline was then applied to ViT-B16 to enable a direct comparison.

## 2. Theoretical Background

### 2.1. Transfer Learning

Transfer learning is a technique that focus on sharing knowledge gain while solving 1 problem and applying to another but related problem. In this way, instead of training from scratch, the pretrained weights are either frozen or fine-tuned on the new dataset. This is especially useful when the target dataset is small, as the model already carries general knowledge about shapes, textures and edges learned from large dataset like ImageNet.

### 2.2. Convolutional Neural Networks ResNet-50

Convolutional Neural Networks process images by applying learnable filters that detect local patterns such as edges, corners and textures. These features are built up layer by layer from simple to complex representations.

ResNet-50 [5] is a 50-layer CNN that introduced residual connections, which allow gradients to flow directly through skip connections during training. This made it possible to train much deeper networks without the vanishing gradient problem. ResNet-50 pretrained on ImageNet produces a 2048-dimensional feature vector from its global average pooling layer, which is used as input to the classification head in this project.

### 2.3. Vision Transformers: ViT-B16

Vision Transformers [6] apply the Transformer architecture, originally developed for natural language processing, directly to images. An input image is divided into fixed size patches of 16x16 pixels. Each patch is flattened and linearly embedded into a vector, and a learnable class token is added to the sequence. Positional embeddings are added to retain spatial information. The sequence is then processed by multiple self-attention layers that capture global relationships between all patches simultaneously.

Vision Transformers have no mechanism to extract the structure about local image. This means they typically need large amounts of data to learn spatial patterns from scratch. However, when pretrained on a large dataset like ImageNet and fine tuned on a smaller one, they can perform well.

## 3. Dataset Selection

Several datasets were evaluated before selecting the final one. The main requirement was enough images per class for transfer learning to work and high enough resolution for 224 by 224 pixels input for the model, so we can create a task good enough to show a meaningful difference between the two architectures.

### 3.1. Oxford Flowers 102

Oxford Flowers 102 [1] was the first choicen dataset. It contains 102 flower categories but only around 10 training images per class according to the official train, validation and test splits by tfds. This is too small for transfer learning experiments to produce reliable results.

### 3.2. CIFAR-100

CIFAR-100 [2] has 100 classes with 500 training images per class, which is a suitable size. However, all images are 32 by 32 pixels. Both ResNet-50 and ViT-B16 expect 224 by 224 input, and upscaling from 32 by 32 makes the images look more blurry and reducing the quality of the comparison. The dataset was rejected due to the low resolution.

### 3.3. PlantVillage

PlantVillage [10] is a plant disease classification dataset with a large number of images per class. The problem was that the task was too easy. The frozen ResNet-50 baseline reached 97 to 98% validation

accuracy within the first few epochs, leaving no room to observe differences between the two architectures. The dataset was rejected for this reason.

### 3.4. Oxford-IIIT Pets

The Oxford-IIIT Pets dataset [3] contains 37 pet breed categories with around 200 images per class. Models converged too quickly on this dataset. The accuracy of the model goes above 90% in first few epochs . Therefore, there was no room for improvement for model due to which dataset was rejected.

### 3.5. Food-101

Food-101 [4] was selected as the final dataset. It provides 750 training images and 250 validation images per class across 101 food categories, which was suitable scale for transfer learning. The images had full resolution and visually diverse food types, making the classification task visually distinct enough to challenge both models.

The dataset is available through TensorFlow Datasets [9], which provides official splits for training and validation.

## 4. Dataset Analysis

### 4.1. Food-101 Structure

Food-101 [4] contains 101 food categories with 750 training images and 250 validation images per class, giving a total of 75,750 training images and 25,250 validation images. All images are in full resolution and stored in RGB format. The dataset is loaded using TensorFlow Datasets [9], where the training split is stored across 32 shards and the validation split across 16 shards. Images within each shard are distributed uniformly across all classes.

### 4.2. Class Selection

10 classes were needed for the comparison, selecting the right ones required careful consideration. Three attempts were made before reaching the final selection.

**Attempt 1 First 20 alphabetical classes:** The simplest approach was to take the first 20 classes alphabetically (label indices 0 to 19). This turned out to be a poor choice because 7 out of 20 classes are visually similar brown baked desserts, including apple_pie, baklava, beignets, bread_pudding, cannoli, carrot_cake, and cheesecake.

**Attempt 2 Every 10th class:** The second attempt selected one class every 10 indices across the full range of 101 classes. This gave a more diversity but still contained some confusable pairs such as risotto and lobster_bisque, which both appear as creamy white dishes, and red_velvet_cake and carrot_cake, which look nearly identical as slices.

**Final selection 10 visually distinct classes:** The final 10 classes were selected manually based on maximum visual distinctiveness in shape, colour and texture. The selected classes are listed in Table 1.

| Index | Class Name |
|---|---|
| 0 | apple_pie |
| 29 | cup_cakes |
| 40 | french_fries |
| 53 | hamburger |
| 58 | ice_cream |
| 63 | macarons |
| 76 | pizza |
| 81 | ramen |
| 95 | sushi |
| 100 | waffles |

**Table 1.** Final 10 selected classes from Food-101

### 4.3. Class Distribution

Each of the 10 selected classes has exactly 750 training images and 250 validation images, making the dataset balanced across all classes.

## 5. Data Preprocessing

### 5.1. Dataset Split

To create a proper three tier split, the validation split was divided into two parts with proportion of 40% and 60%. The final split used in this project is as follows:

| Split | Source | Images | Per Class |
|---|---|---|---|
| Training | train 100% | 7,500 | 750 |
| Validation | validation 40% | 1,000 | 100 |
| Test | validation 60% | 1,500 | 150 |

**Table 2.** Dataset split used for all experiments

The dataset was loaded with `shuffle_files=False` to ensure a fixed and non-overlapping between splits across all runs. The official validation split in Food-101 contains exactly 250 images per class, so we did it into 2 parts producing a perfectly balanced validation and test set of 125 images per class each.

### 5.2. Label Remapping

The 10 selected classes have original Food-101 label indices of 0, 29, 40, 53, 58, 63, 76, 81, 95 and 100. These were remapped to consecutive indices 0 to 9 using a `StaticHashTable` in TensorFlow [8]. This is required because the classification head outputs 10 values and expects labels in the range 0 to 9. [1]

### 5.3. Training Pipeline

The augmentation steps applied during the training includes:

- Resize to 224 by 224 pixels
- Random horizontal flip
- Random brightness adjustment (max delta = 0.1)
- Random contrast adjustment (range 0.9 to 1.1)
- ResNet-50 normalisation using `preprocess_input` [8]

Each step introduces controlled variation to reduce overfitting on the small dataset. Images were shuffled with a buffer size of 2,000 and grouped into batches of 64. Additional preprocessing was applied to the data called preprocess input which was imported from tensorflow.keras.applications.resnet50 [8] and converts images from RGB to BGR format and subtracts the ImageNet mean pixel values from each channel, which is the exact normalisation that ResNet-50 was trained with.

### 5.4. Validation and Test Pipeline

No augmentation was applied to the validation and test sets. Both use the same preprocessing: resize to 224 by 224 pixels followed by ResNet-50 preprocess input function and batches of 64 were used without dropping the remainder.

## 6. Model Architecture

### 6.1. Shared Backbone ResNet-50

ResNet-50 [5] pretrained on ImageNet was used as the backbone for all CNN experiments. The top layer was removed and the backbone was frozen means keeping its weights constants during training. The backbone takes a (224,224,3) image as input and produces a 2048 dimensional feature vector after Global Average Pooling, which is then passed to the classification head.

[1] The implementation of the label remapping function was developed with the assistance of an AI tool.

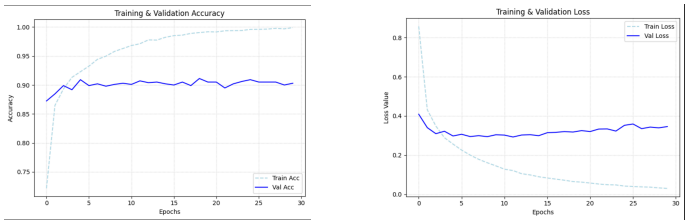



**Figure 1.** Accuracy and Loss curves for M1 baseline model

```python
resnet50_base = tf.keras.applications.ResNet50(
                weights='imagenet',
                include_top=False,
                input_shape=(IMG_SIZE, IMG_SIZE, 3))
resnet50_base.trainable = False

print(f"{'Layer (type)':<25} | {'Output Shape':<20} | {'Params':<10}")
print("-" * 65)

# First 10 layers of ResNet_50
for layer in resnet50_base.layers[:10]:
    name = layer.name
    shape = str(layer.output.shape)
    params = f"{layer.count_params():,}"
    print(f"{name:<25} | {shape:<20} | {params:<10}")
```

**Code 1.** Code example of Resnet-50 backbone

### 6.2. Classification Head Variants

Five classification head variants were trained on top of the frozen backbone. Each model adds one regularisation technique on top of the previous one, allowing the effect of each technique to be observed in isolation. LeakyReLU with a negative slope of 0.01 was used as the activation function for all hidden layers to avoid the dying Relu problem.

| Model | Architecture | Added |
|---|---|---|
| M1 | GAP → Dense(10) | Baseline |
| M2 | GAP → DNN (1024→512→256) | Deep head |
| M3 | GAP → DNN + Dropout | Dropout |
| M4 | GAP → DNN + BN + Dropout | Batch Normalisation |
| M5 | GAP → DNN + BN + Dropout + L2 | L2 regularisation |
| M6 | M5 + unfrozen backbone | Fine-tuning |

**Table 3.** Classification head variants trained on frozen ResNet-50

Models M1 to M5 share the same deep neural network structure of three dense layers with sizes 1024, 512 and 256, followed by a final Dense(10) layer with softmax activation. By hyperparameter tunning of learning rate, L2 regularisation and droupouts 3 variants of Model 5 were created and the best one was choosen for fine tuning.

### 6.3. Fine-Tuning Model 5

After getting the best variant from model M5, version 5.3 training with the frozen backbone, the backbone was partially unfrozen for fine-tuning. Resnet-50 contains 175 Layers in total and out of which 0 to 139 were kept frozen. Layers 140 and above 35 in total were made unfrozen and trained with a very small learning rate of 1e-5 (0.00001) to avoid destroying the pretrained weights. Training continued from initial epoch of 100 for additional 30 epochs using early stopping with a patience of 5 on validation loss.

```python
# --- MODEL 5.3: ResNet & DNN : Dropout + Batch Normalization + Regularisation ---

resnet50_base.trainable = False

model_DNN_DO_BN_L2_V3 = tf.keras.Sequential()
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.Input(
    shape=(IMG_SIZE, IMG_SIZE, 3)))

model_DNN_DO_BN_L2_V3.add(resnet50_base)
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    GlobalAveragePooling2D())

model_DNN_DO_BN_L2_V3.add(tf.keras.layers.Dense
    (1024, use_bias=False, kernel_regularizer=l2
    (1e-4)))
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    BatchNormalization())
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    LeakyReLU(negative_slope=0.01))
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    Dropout(0.5))

model_DNN_DO_BN_L2_V3.add(tf.keras.layers.Dense
    (512, use_bias=False, kernel_regularizer=l2
    (1e-4)))
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    BatchNormalization())
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    LeakyReLU(negative_slope=0.01))
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    Dropout(0.4))

model_DNN_DO_BN_L2_V3.add(tf.keras.layers.Dense
    (256, use_bias=False, kernel_regularizer=l2(
     1e-4)))
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    BatchNormalization())
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    LeakyReLU(negative_slope=0.01))
model_DNN_DO_BN_L2_V3.add(tf.keras.layers.
    Dropout(0.2))

model_DNN_DO_BN_L2_V3.add(tf.keras.layers.Dense(
    NUM_CLASSES, activation='softmax'))

model_DNN_DO_BN_L2_V3.compile(
    optimizer= tf.keras.optimizers.Adam(
    learning_rate=1e-5),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model_DNN_DO_BN_L2_V3.summary()
```

**Code 2.** Code example of M5.3 model

## 7. Results ResNet-50

### 7.1. M1 Baseline

The simplest possible head a single Dense(10) layer placed directly on the Global Average Pooling output.

- **Validation:** Best validation accuracy was 91.12%. Training accuracy reached 99.91%.
- **Loss:** Validation loss started rising in the early epochs and

Model was clearly overfitting over the training data.

### 7.2. M2 Deep Neural Network Head

A three layered, MLP network (1024→512→256) was added to give the model to train on the specific food101 dataset for choosen 10 classes .

- **Validation:** Accuracy reached 91.22%, in very few epochs.
- **Loss:** Validation loss rose was 0.33 with this accuracy.

Model was overfitting from the start of the training.

### 7.3. M3 DNN with Dropout

In order to reduce, the overfitting Dropout was used for regularisation technique. Dropout layers were added after each dense layer with probabilities of 0.4, 0.3 and 0.2 to prevent the model from memorising the training data.

- **Validation:** Accuracy improved to 92.24%.
- **Loss:** Validation loss stayed around 0.31 but still showed a rising trend.

Dropout seem to improve the overfitting problem but still the model overfitted over epochs.

### 7.4. M4 DNN with Batch Normalisation and Dropout

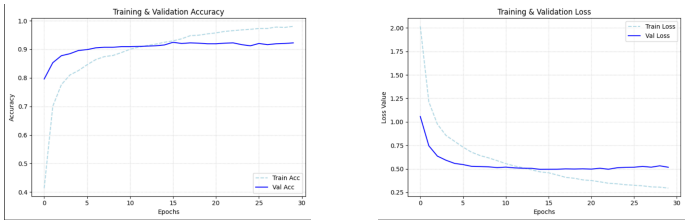
Batch Normalisation was added before each activation function to stabilise training by normalising the layer inputs.

- **Validation:** Accuracy reduced to 91.94%.
- **Loss:** Validation loss reduced to 0.28.

Batch Normalisation clearly stabilise the training and gradients updated in smooth way compared to M2 and M3, by observing the curves.

### 7.5. M5 DNN with BN, Dropout and L2 Regularisation

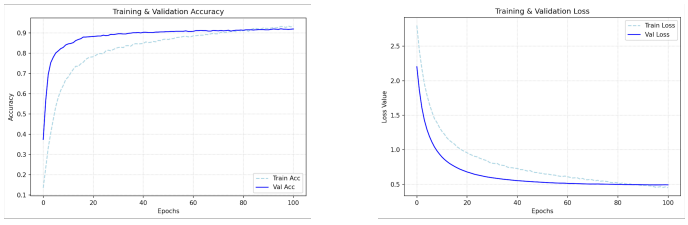
To apply a proper regularisation technique, the L2 Regularisation was added to all dense layers. Three variants with different L2 strengths and learning rates were tested and the one with the most stable training curves was selected for fine-tuning, which turned out to be the M5.3




Accuracy Loss

**Figure 2.** M5.2 accuracy and loss curves

- **Validation:** Best validation accuracy achieved was 92.04%.
- **Loss:** Validation loss was 0.49. This was the most stable loss curve of all frozen models. The high loss value compared to other models is because Keras includes the L2 penalty in the reported loss.
  The model M5.3 had a bit low accuracy than M5.1 of 92.45% and has the same loss value of 0.49 but he the model 5.3 was choosen for the fine tuning because the training and validation curves show the model M5.3 generalised in a good way and do not overfit at all.




Accuracy Loss

**Figure 3.** M5.3 Accuracy and Loss curves

### 7.6. M6 Fine-Tuning

The best M5 variant, M5.3 was taken for fine-tunning. ResNet-50 layers 140 to 175 were unfrozen and trained with a learning rate of 1e-5. Early stopping was used with a patience of 5 on validation loss to avoid overfitting during fine-tuning.

- **Validation:** Accuracy improved from 92.04% to 92.94% after 6 epochs only, fine tuning did not improved the accuracy of the model because the classification MLP head attached had above 2 millions parameters, leaving less room of improvement in fine-tuning of ResNet-50 base.
- **Loss:** Validation loss remained the same of 0.49 in both cases

| Model | Architecture | Val Acc | Val Loss | Loss Trend | Outcome |
|---|---|---|---|---|---|
| M1 | Baseline | 91.12% | 0.29 | Rising | Overfitting |
| M2 | DNN | 91.22% | 0.33 | Rising | Overfitting |
| M3 | DNN + Dropout | 92.24% | 0.31 | Rising | Overfitting |
| M4 | DNN + BN + Dropout | 91.94% | 0.28 | Slowly rising | Mild overfitting |
| M5 | DNN + BN + DO + L2 | 92.04% | 0.49* | Stable | Stable training |
| M6 | Fine-tuned | **92.94%** | 0.49* | Stable | Stable training |

*M5 and M6 loss includes L2 regularisation penalty.

**Table 4.** ResNet-50 results summary across all six models

### 7.7. ResNet-50 Summary

| Food | Precision | Recall | F1-Score |
|---|---|---|---|
| apple_pie | 0.83 | 0.83 | 0.83 |
| cup_cakes | 0.95 | 0.94 | 0.94 |
| french_fries | 0.96 | 0.96 | 0.96 |
| hamburger | 0.95 | 0.92 | 0.93 |
| ice_cream | 0.94 | 0.90 | 0.92 |
| macarons | 0.96 | 0.96 | 0.96 |
| pizza | 0.97 | 0.99 | **0.98** |
| ramen | 0.99 | 0.97 | **0.98** |
| sushi | 0.90 | 0.93 | 0.91 |
| waffles | 0.87 | 0.90 | 0.88 |
| **Overall Accuracy** | | | **0.93** |

**Table 5.** Perclass classification report on test set for M6 ResNet-50 fine-tuned

Overall, ResNet-50 truned out to be very powerfull for small datasets and overfit the data very quickly. By adding regularisation techniques one by one helped reduce overfitting and can help model generalises in a proper way. Adding a really big muliple layer perceptron head, can make the model solve specific classification problem with a very high accuracy but at the same time still increasing the complexity of the model.The per-class results on the test set are shown in Table 5.

## 8. Vision Transformer: ViT-B16

ViT-B16 [6] was loaded with ImageNet pretrained weights using keras_hub [7] and frozen in the same way as ResNet-50. Unlike ResNet-50 which outputs a 2048 dimensional feature map that requires Global Average Pooling, the ViT backbone outputs 197 vectors of 768 dimensions each one for each of the 196 image patches plus one special learnable token called the CLS token. It is extracted using a Lambda layer and passed directly to the classification head.

Different type of preprocessing from ResNet-50 was applied for ViT-B16. Images were first scaled to the range [0, 1] and then normalised using the ImageNet mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225] per channel. GELU activation was

used in all hidden layers of the ViT heads instead of LeakyReLU, as GELU is the activation used inside the ViT transformer blocks themselves, making it more consistent with the pretrained backbone.

Five classification head variants were trained on top of the frozen ViT backbone following the same progression as ResNet-50.

| Model | Architecture | Added |
|---|---|---|
| V1 | CLS → Dense(10) | Baseline |
| V2 | CLS → DNN (512→256) | Deep head, GELU |
| V3 | CLS → DNN + Dropout | Dropout |
| V4 | CLS → DNN + Dropout + L2 | L2 regularisation |
| V4.2 | CLS → DNN + Dropout + L2 | Hyperparameter tunning |

**Table 6.** Classification head variants trained on frozen ViT-B16

## 9. Results ViT-B16

### 9.1. Overview

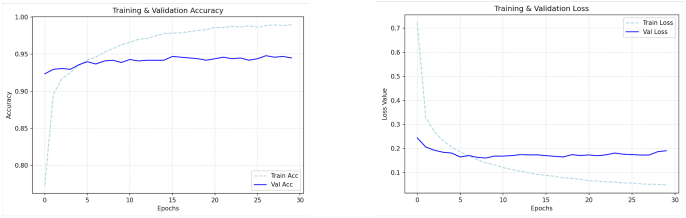
Five classification head variants were trained on the frozen ViT-B16 backbone. All models used Adam optimiser with a learning rate between (1e-3, 1e-5) and batch size of 64. The same 10-class Food-101 dataset and split were used as in the ResNet-50 experiments.

### 9.2. V1 Baseline

The simplest head with a single Dense(10) layer placed directly on the CLS token output.

- **Validation:** Best validation accuracy was 94.79% which was already higher than Resnet-50 base model.
- **Loss:** Validation loss started rising after epoch 20, ending at 0.1907 from a best of 0.1726.

But similarly to M1 of Resnet-50 this model was also clearly overfitting over the training data.




Accuracy Loss

**Figure 4.** V1 baseline accuracy and loss curves

### 9.3. V2 Deep Neural Network Head

A two-layer head (512→256) with GELU activation was added on top of the CLS token.

- **Validation:** Accuracy improved slightly to 95.30%. Similar to the improvement seen with DNN heads in ResNet.
- **Loss:** Validation loss rose from 0.1726 to 0.2264

Increasing more parameters made overfitting worse, model becomes more complex.

### 9.4. V3 DNN with Dropout

Dropout layer was added after each dense layer with probalilities of 0.4 and 0.3.

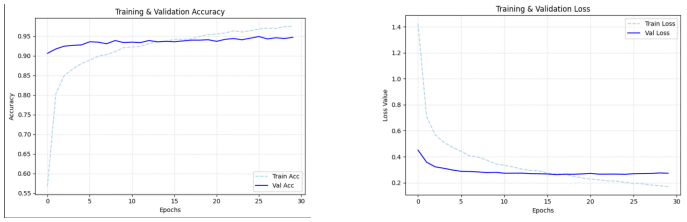
- **Validation:** Best validation accuracy reached 95.51% at epoch 25, the highest among all frozen ViT variants.
- **Loss:** Validation also reduced from 0.2264 to 0.1620, very small increase at the end of the training. This was the most stable loss curve among the previous two variants.

However, model started to overfit again as the training goes on.

### 9.5. V4 DNN with Dropout and L2 Regularisation

L2 regularisation (1e-4) was added to all dense layers alongside higher Dropout rates of 0.5 and 0.4.

- **Validation:** Accuracy dropped slightly to 94.89% compared to V3. The L2 penalty slowed down learning.
- **Loss:** Validation loss was reducing in a stable way reaching at 0.2689 againset best validation accuracy.



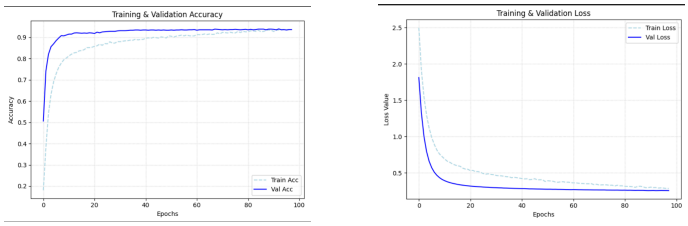

Accuracy Loss

**Figure 5.** V4 accuracy and loss curves

### 9.6. V4.2 Hyperparameter Tunning

The same architecture as V4 but with adjusted Dropout rates (0.5 and 0.2), learning rate of 1e-5 and early stopping with a patience of 5 on validation loss. The model was allowed to train for up to 130 epochs.

- **Validation:** The model trained for 98 epochs before early stopping triggered. Best validation accuracy was 93.97% at epoch 93. The training accuracy (93.49%) stayed very close to the validation accuracy, showing a smooth training.
- **Loss:** The most stable loss curve of all ViT variants. Validation loss remained almost same 0.2560. This model was selected as the final ViT model and saved for test evaluation.




Accuracy Loss

**Figure 6.** V4.2 accuracy and loss curves

### 9.7. ViT-B16 Summary

| Model | Architecture | Val Acc | Val Loss | Loss Trend | Outcome |
|---|---|---|---|---|---|
| V1 | Baseline | 94.79% | 0.1726 | Rising | Overfitting |
| V2 | DNN | 95.30% | 0.2264 | Rising | Overfitting |
| V3 | DNN + Dropout | **95.51%** | **0.1620** | Stable | Stable training |
| V4 | DNN + Dropout + L2 | 94.89% | 0.2689 | Stable | Stable training |
| V4.2 | V4 + Hyperparameter tunning | 93.97% | 0.2560 | Stable | Best generalisation |

*V4 and V4.2 loss includes L2 penalty.

**Table 7.** ViT-B16 results summary across all five variants

Overall, ViT-B16 started at a much higher baseline accuracy than ResNet-50 (94.80% vs 91.12%) without any regularisation techniquw. The DNN head variants showed smaller improvements than in ResNet. V3 achieved the best validation accuracy at 95.51% while V4.2 produced the most stable and well generalised training behaviour. The per class results on the test set are shown in Table 8.

| Food | Precision | Recall | F1-Score |
|---|---|---|---|
| apple_pie | 0.89 | 0.89 | 0.89 |
| cup_cakes | 0.95 | 0.98 | 0.97 |
| french_fries | 0.97 | 0.98 | 0.97 |
| hamburger | 0.95 | 0.94 | 0.94 |
| ice_cream | 0.96 | 0.93 | 0.94 |
| macarons | 0.99 | 0.97 | 0.98 |
| pizza | 0.97 | 0.99 | **0.98** |
| ramen | 0.99 | 1.00 | **1.00** |
| sushi | 0.97 | 0.97 | 0.97 |
| waffles | 0.92 | 0.90 | 0.91 |
| **Overall Accuracy** | | | **0.95** |

**Table 8.** Perclass classification report on test set for ViT-B16 V4.2

## 10. Conclusion

This project compared ResNet-50 and ViT-B16 for image classification on a 10-class subset of Food-101 using transfer learning from ImageNet. The aim was to investigate how these two architectures perform on a small dataset using the transfer learning from ImageNet.

The original ViT paper [6] states that Vision Transformers underperform CNNs on small datasets when trained from scratch, due to the absence of CNN inductive biases such as locality and translation invariance properties. However, this project studied the scenario of transfer learning from ImageNet pretrained weights for small dataset. So, the ViT paper states that ViT outperforms CNN based models when pretrained on large datasets and the results of this porject are consistent with this statement, ViT-B16 achieved 95% test accuracy compared to 93% for ResNet-50 on the same task, confirming that the of ViT are more efficient than CNN in the transfer learning method. [2]

- **Classfication Head:** It was clearly observed for small datasets like Food101 having 750 images per class both of these models classification was not so difficult and their cooresonding architectures with the ImageNet weight can easily solve such kind of small scale problems however, leaving some room for increasing their performance in a specific domain in the form of better generalisation. For such model adding MLP as a head no doubt increased the complexity of the model, leading to overfitting of the models but the same time it can help models to to slove the specific tasks with highest possible accuracy.
- **Training Stability:** Due to millions of parameters, both models learn very quickly on small dataset due to which a steady increase in accuracy and steady decrease in the loss curves were observed. So, in order to make the training hard to the model different regularisation techniques were applied periodically in order to achieve the best possible modles from both of these by making them learning in a nice gradually way.
- **ImageNet Class Overlap:** A potential limitation of this study is that ImageNet contains some food related categories that overlap with the selected dataset of Food101. Specially, pizza and ice cream which also adds to the high accuracy in the early stages of the trainings for both Renet and ViT based models.

Future work can be evaluating both architectures on datasets with no overlap with ImageNet classes at all to investigate their performances furthermore.

[2] Source code, trained models and experiment results are available at: https://git.oth-aw.de/d801/deep-vision-models

## References


[1] Visual Geometry Group, University of Oxford, *The oxford 102 flower dataset*, 2008. [Online]. Available: https://www.robots.ox.ac.uk/~vgg/data/flowers/102/

[2] A. Krizhevsky, *Cifar-100 dataset*, 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html

[3] Visual Geometry Group, University of Oxford, *The oxford-iiit pet dataset*, 2012. [Online]. Available: https://www.robots.ox.ac.uk/~vgg/data/pets/

[4] Computer Vision Laboratory, ETH Zurich, *Food-101 dataset*, 2014. [Online]. Available: https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[6] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale", in *International Conference on Learning Representations (ICLR)*, 2021.

[7] Keras, *Keras api reference*, 2024. [Online]. Available: https://keras.io/api/

[8] TensorFlow, *Tensorflow api reference*, 2024. [Online]. Available: https://www.tensorflow.org/api_docs

[9] TensorFlow, *Tensorflow datasets*, 2024. [Online]. Available: https://www.tensorflow.org/datasets

[10] PlantVillage Project, Penn State University, *Plantvillage dataset*. [Online]. Available: https://plantvillage.psu.edu/